

Live WebRTC — ICE Fix (App Prompt)

Live WebRTC — ICE Connection Failed

Complete App Fix Prompt (Controller + Player)D

D

****Backend + TURN server already verified. Problem is in Android app code.**** D

****Copy the "Cursor Prompt" section below into Cursor inside your App repository.****␣

D

---D

D

```
## Verified Server Status (DO NOT change backend)D
```

D

	Check	Result	Đ
--	-------	--------	---

$$| \text{---} | \text{---} | \mathbb{D}$$

| GET `/api/live/config` | Returns STUN + TURN with username/credential | D

| coturn service | `active (running)` | 0

| Firewall 3478 UDP/TCP | Open | 0

```
| VPS env | TURN_URL, TURN_USERNAME, TURN_PASSWORD set | D
```

D

****coturn journalctl during live test shows ONLY systemd start/stop — NO allocation logs.**** ③

This proves: ****Android apps are NOT connecting to TURN server.**** ☺

Fix = app must use `ice\_servers` from API + complete ICE candidate exchange.

D

---D

D

Symptoms

D

- Controller UI: ****"ICE Connection Failed"****Ð

- Player: no live audio on speaker

- Backend may show `offer\_received` but ICE never completes

- coturn log: silent during live test (no `allocation` / `session` entries)D

D

---D

D

Root Causes (App Side — fix ALL of these)D

D

1. ****Hardcoded STUN**** — app ignores API ``ice_servers`` (TURN never used)🔻

2. ****Missing ICE exchange**** — no `POST/GET /api/live/ice` on one or both apps

3. ****Player not handling WS events**** — misses ``live_session`` / ``live_offer`` ⚠

4. ****Wrong Gson field**** — API sends `credential`, app reads `password` ! TURN auth fails

5. ****Candidates before remote SDP**** — dropped, never added

6. ****Player WebSocket offline**** — `player\_notified sent=0` in backend logs

7. ****No ICE polling**** — only POST, never GET peer candidates

D

---D

D

[illegible]

CURSOR PROMPT — COPY FROM HERED

%P%

D

```

Fix Live WebRTC "ICE Connection Failed" in this Android app.

CONTEXT:

- Backend: <https://api.shikkhasomoy.com>
- Server TURN is working: turn:194.233.86.3:3478 user=schoolbell
- GET /api/live/config returns ice\_servers with TURN — verified
- coturn runs on VPS but shows NO allocation logs during live test
- Therefore this app is NOT using TURN and/or NOT exchanging ICE candidates

TASK:

1. Search entire codebase for: PeerConnection, WebRtc, live, ice\_servers, IceCandidate, live\_session, live\_offer, live\_answer, live\_ice, stun.l.google.com
2. List every file involved in live announcement
3. Fix ALL issues below — implement missing parts completely
4. Add Log.d("LiveWebRTC", ...) on every critical step
5. Do NOT change unrelated features (bell, azan, schedules, announcements)

MANDATORY FIXES:

A) ice\_servers FROM API — never hardcode STUN

- Controller: use ice\_servers from POST /api/live/session response
- Player: use ice\_servers from WS live\_session event (store it!)  
fallback: GET /api/live/config
- Map API field "credential" to PeerConnection IceServer setPassword()
- Log ice\_servers JSON before creating PeerConnection — must include turn: URL

B) ICE candidate exchange — BOTH directions, BOTH apps

- On each local candidate: POST /api/live/ice  
body: { session\_id, role, candidate, sdpMid, sdpMLineIndex }
- On WS type=live\_ice OR after offer/answer: poll GET /api/live/ice every 500ms for 30s
- For each returned candidate: peerConnection.addIceCandidate()
- Log: "ICE POST sent" and "ICE GET received N candidates"

C) Candidate queue — flush after setRemoteDescription

- Queue remote candidates if remote SDP not set yet
- After setRemoteDescription success: add all queued candidates

D) Complete signaling flow

Controller: session ! PeerConnection ! mic track ! offer ! POST offer  
! wait answer ! setRemoteDescription ! ICE exchange  
Player: live\_session (store ice\_servers) ! live\_offer ! PeerConnection  
! setRemoteDescription ! answer ! POST answer ! ICE exchange  
! onTrack ! speaker ON, auto play

E) WebSocket (Player especially)

- Connect: <wss://api.shikkhasomoy.com/ws/announce?token=JWT&role=user>
- Handle JSON: live\_session, live\_offer, live\_ice, live\_end
- Reconnect on disconnect — foreground service must keep WS alive
- Remove ALL old binary PCM audio over WebSocket code

## F) PeerConnection config

- sdpSemantics = UNIFIED\_PLANB
- continualGatheringPolicy = GATHER\_CONTINUALLYB
- iceTransportsType = ALL (not RELAY only, not NONE)B

D

## DELIVERABLES:Đ

- LiveApiService.kt with all /api/live/\* endpoints
- LiveWebRtcController.kt OR LiveWebRtcPlayer.kt (depending on this repo)
- Updated WebSocket handler
- Debug logs proving TURN in ice\_servers and ICE POST/GET working

D

SUCCESS: Controller shows "Live", Player hears mic on speaker, D

coturn journalctl shows allocation logs during test.Đ

“D

D

# %P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%

```
END CURSOR PROMPT
```

# %P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%P%

D

---D

D

## ## Diagnosis Decision TreeD

D

“D

ICE Connection Failedð

% D

% % coturn log silent during live test?Đ

% % YES !' App not using TURN (hardcoded STUN or wrong ice servers)D

% D

```
% % backend: player_notified sent=0?Đ
```

```
% % YES !' Player WebSocket offline — fix WS in player appD
```

% D

```
% % backend: offer received but NO session connected?D
```

```
% % YES !' Player not sending answer — fix live offer handlerD
```

 $\% \text{ D}$ 

```
% % backend: session connected but ICE fails?D
```

```
% % YES !' ICE POST/GET broken — fix Step ICE exchange below
```

% D

% % same WiFi works, different network fails?D

## % % TURN not used in app — fix ice servers from API

 ${}^{\prime\prime}\mathcal{D}$ 

D

---D

D

## ## Fix A — ice servers from API (CRITICAL)D

D

### ### WRONGÐ

```
```kotlin
```

```
PeerConnection.IceServer.builder("stun:stun.l.google.com:19302").createIceServer()
```

 $\mathbb{D}$

D

```

#### CORRECT — shared helper (put in both apps)
```kotlin
object LiveWebRtcConfig {
 fun buildIceServers(servers: List<IceServerDto>): List<PeerConnection.IceServer> {
 require(servers.isNotEmpty()) { "ice_servers empty — fetch from API first" }
 Log.d("LiveWebRTC", "ice_servers count=${servers.size}: $servers")

 return servers.map { s ->
 val builder = PeerConnection.IceServer.builder(s.urls)
 if (!s.username.isNullOrBlank()) builder.setUsername(s.username)
 // API field is "credential" NOT "password"
 if (!s.credential.isNullOrBlank()) builder.setPassword(s.credential)
 builder.createIceServer()
 }
 }

 fun createRtcConfig(servers: List<IceServerDto>): PeerConnection.RTCConfiguration {
 return PeerConnection.RTCConfiguration(buildIceServers(servers)).apply {
 sdpSemantics = PeerConnection.SdpSemantics.UNIFIED_PLAN
 continualGatheringPolicy = PeerConnection.ContinualGatheringPolicy.GATHER_CONTINUALLY
 iceTransportsType = PeerConnection.IceTransportsType.ALL
 }
 }
}

Gson model — field names MUST match API
```kotlin
data class IceServerDto(
    val urls: String,
    val username: String? = null,
    val credential: String? = null // NOT "password" — API sends "credential"
)

data class LiveConfigResponse(
    @SerializedName("ice_servers") val iceServers: List<IceServerDto>
)

data class CreateSessionResponse(
    @SerializedName("session_id") val sessionId: String,
    @SerializedName("ice_servers") val iceServers: List<IceServerDto>,
    val status: String
)

#### Where to get ice_servers
| App | Source |
|-----|-----|
| Controller | `POST /api/live/session` ! response.iceServers |
| Player | WS `live_session` ! store in memory ! use on `live_offer` |

```

| Player fallback | `GET /api/live/config` if live_session missed |

****NEVER create PeerConnection before ice_servers is available.****

Fix B — ICE Candidate Exchange (MOST COMMON BUG)

Backend stores candidates in memory. Each app must POST own candidates and GET peer's.

POST — on every local candidate

```kotlin

// In PeerConnection.Observer.onIceCandidate

override fun onIceCandidate(candidate: IceCandidate?) {

candidate ?: return

scope.launch {

try {

liveApi.postIce(IceRequest{

sessionId = sessionId,

role = myRole, // "controller" or "player"

candidate = candidate.sdp,

sdpMid = candidate.sdpMid,

sdpMLineIndex = candidate.sdpMLineIndex

})

Log.d("LiveWebRTC", "ICE POST ok: \${candidate.sdp.take(60)}...")

} catch (e: Exception) {

Log.e("LiveWebRTC", "ICE POST failed", e)

}

}

}

```

GET — poll peer candidates

```kotlin

private var icePollJob: Job? = null

fun startIcePolling(sessionId: String, role: String) {

icePollJob?.cancel()

icePollJob = scope.launch {

repeat(60) { // 500ms x 60 = 30 seconds

delay(500)

try {

val resp = liveApi.getIceCandidates(sessionId, role)

Log.d("LiveWebRTC", "ICE GET: \${resp.candidates.size} candidates")

resp.candidates.forEach { c ->

addRemoteCandidate(c.sdpMid, c.sdpMLineIndex, c.candidate)

}

} catch (e: Exception) {

Log.e("LiveWebRTC", "ICE GET failed", e)

}

}

```

 }
}
}
}
fun stopIcePolling() {
 icePollJob?.cancel()
 icePollJob = null
}
```
}
### Trigger polling on:
- After `setRemoteDescription` succeeds ! `startIcePolling(sessionId, myRole)`
- WebSocket `{ "type": "live_ice", "session_id": "..." }` ! poll once or restart loop
}
### GET role parameter = YOUR role (backend returns PEER's candidates)
```
Controller polls: GET /api/live/ice?session_id=X&role=controller ! gets player candidates
Player polls: GET /api/live/ice?session_id=X&role=player ! gets controller candidates
```
}
---
}
## Fix C — Candidate Queue (timing bug)
}
```kotlin
private val pendingRemoteCandidates = mutableListOf<Triple<String?, Int, String>>()
private var remoteDescriptionSet = false
}
fun addRemoteCandidate(sdpMid: String?, sdpMLineIndex: Int, candidate: String) {
 if (remoteDescriptionSet && peerConnection != null) {
 peerConnection!!.addIceCandidate(IceCandidate(sdpMid, sdpMLineIndex, candidate))
 Log.d("LiveWebRTC", "addIceCandidate: $candidate")
 } else {
 pendingRemoteCandidates.add(Triple(sdpMid, sdpMLineIndex, candidate))
 Log.d("LiveWebRTC", "queued candidate (remote SDP not ready)")
 }
}
}
}
}
private fun flushPendingCandidates() {
 remoteDescriptionSet = true
 pendingRemoteCandidates.forEach { (mid, idx, cand) ->
 peerConnection?.addIceCandidate(IceCandidate(mid, idx, cand))
 }
 pendingRemoteCandidates.clear()
}
}
}
// Call flushPendingCandidates() inside setRemoteDescription onSuccess callback
```
}
---
}
## Fix D — Controller Complete Flow

```

```

}
```kotlin
class LiveWebRtcController {
 private val context: Context,
 private val liveApi: LiveApiService,
 private val scope: CoroutineScope
} {
 private var peerConnection: PeerConnection? = null
 private var sessionId: String? = null
 private var iceServers: List<IceServerDto> = emptyList()

 suspend fun startLive(): Result<Unit> = runCatching {
 // 1. Session + ice_servers
 val session = liveApi.createSession(CreateSessionRequest(role = "controller"))
 sessionId = session.sessionId
 iceServers = session.iceServers
 Log.d("LiveWebRTC", "session=${session.sessionId} servers=${iceServers}")

 // 2. PeerConnection
 initPeerConnection()

 // 3. Mic track
 val audioSource = factory.createAudioSource(MediaConstraints())
 val localTrack = factory.createAudioTrack("mic", audioSource)
 peerConnection!!.addTrack(localTrack, listOf("stream"))

 // 4. Offer
 val offer = suspendCoroutine<SessionDescription> { cont ->
 peerConnection!!.createOffer(object : SdpObserverAdapter() {
 override fun onCreateSuccess(sdp: SessionDescription?) {
 sdp?.return
 peerConnection!!.setLocalDescription(object : SdpObserverAdapter() {
 override fun onSetSuccess() { cont.resume(sdp) }
 }, sdp)
 }
 }, MediaConstraints())
 }
 liveApi.postOffer(OfferRequest(sessionId!!, "controller", offer.toDto()))
 Log.d("LiveWebRTC", "offer posted")
 }

 fun onLiveAnswer(answer: SessionDescription) {
 peerConnection?.setRemoteDescription(object : SdpObserverAdapter() {
 override fun onSetSuccess() {
 flushPendingCandidates()
 startIcePolling(sessionId!!, "controller")
 Log.d("LiveWebRTC", "remote answer set, ICE polling started")
 }
 }, answer)
 }
}

```

```

// Observer — log ICE state
private inner class PcObserver : PeerConnection.Observer {
 override fun onIceConnectionChange(state: PeerConnection.IceConnectionState) {
 Log.d("LiveWebRTC", "ICE state: $state")
 when (state) {
 CONNECTED, COMPLETED -> onUiState("Live")
 FAILED -> onUiState("ICE Connection Failed")
 DISCONNECTED -> onUiState("Disconnected")
 else -> onUiState("Connecting...")
 }
 }
 override fun onIceCandidate(c: IceCandidate?) { /* POST /api/live/ice */ }
 // ... implement other required methods (can be empty)
}

```kotlin
when (type) {
    "live_answer" -> controller.onLiveAnswer(parseAnswer(json))
    "live_ice" -> controller.startIcePolling(sessionId, "controller")
    "live_end" -> controller.stopLive()
}

```
Fallback if WS misses answer: poll `GET /api/live/session?role=controller&session_id=X`

Fix E — Player Complete Flow

```kotlin
class LiveWebRtcPlayer {
    private val context: Context
    private val liveApi: LiveApiService
    private val scope: CoroutineScope
} {
    private var sessionId: String? = null
    private var iceServers: List<IceServerDto> = emptyList()
    private var peerConnection: PeerConnection? = null

    // Step 1 — on WebSocket live_session (BEFORE live_offer!)
    fun onLiveSession(id: String, servers: List<IceServerDto>) {
        sessionId = id
        iceServers = servers
        Log.d("LiveWebRTC", "live_session stored: $id servers=$servers")
    }

    // Step 2 — on WebSocket live_offer
    suspend fun onLiveOffer(offer: SessionDescription) {

```

```

        if (iceServers.isEmpty()) {
            // fallback
            iceServers = liveApi.getConfig().iceServers
            Log.w("LiveWebRTC", "ice_servers from /config fallback")
        }
        initPeerConnection(iceServers)
    }

    peerConnection!!.setRemoteDescription(object : SdpObserverAdapter() {
        override fun onSetSuccess() {
            flushPendingCandidates()
            createAndPostAnswer()
        }
    }, offer)

    private fun createAndPostAnswer() {
        peerConnection!!.createAnswer(object : SdpObserverAdapter() {
            override fun onCreateSuccess(sdp: SessionDescription?) {
                sdp ?: return
                peerConnection!!.setLocalDescription(object : SdpObserverAdapter() {
                    override fun onSetSuccess() {
                        scope.launch {
                            liveApi.postAnswer(AnswerRequest(sessionId!!, "player", sdp.toDto()))
                            startIcePolling(sessionId!!, "player")
                            Log.d("LiveWebRTC", "answer posted, ICE polling started")
                        }
                    }
                }, sdp)
            }
        }, MediaConstraints())
    }

    // Step 3 — on remote audio track
    override fun onAddTrack(receiver: RtpReceiver?, streams: Array<out MediaStream>?) {
        val track = receiver?.track() as? AudioTrack ?: return
        track.setEnabled(true)
        val am = context.getSystemService(Context.AUDIO_SERVICE) as AudioManager
        am.mode = AudioManager.MODE_IN_COMMUNICATION
        am.isSpeakerphoneOn = true
        Log.d("LiveWebRTC", "remote audio track playing on speaker")
    }

    /** Player WebSocket handler */
    ````kotlin
 when (type) {
 "live_session" -> player.onLiveSession(json.sessionId, json.iceServers)
 "live_offer" -> scope.launch { player.onLiveOffer(parseOffer(json)) }
 "live_ice" -> player.startIcePolling(sessionId, "player")
 "live_end" -> player.stopLive()
 }

```

```

 "new_announcement" -> /* KEEP existing handler — do not remove */
}
```


**Player does NOT call POST /api/live/session.**

---

## Fix F — Retrofit LiveApiService (both apps)

```kotlin
interface LiveApiService {
 @GET("api/live/config")
 suspend fun getConfig(): LiveConfigResponse

 @POST("api/live/session")
 suspend fun createSession(@Body body: CreateSessionRequest): CreateSessionResponse

 @GET("api/live/session")
 suspend fun getSession(
 @Query("session_id") sessionId: String,
 @Query("role") role: String
): SessionResponse

 @POST("api/live/offer")
 suspend fun postOffer(@Body body: OfferRequest): ApiMessageResponse

 @POST("api/live/answer")
 suspend fun postAnswer(@Body body: AnswerRequest): ApiMessageResponse

 @POST("api/live/ice")
 suspend fun postIce(@Body body: IceRequest): ApiMessageResponse

 @GET("api/live/ice")
 suspend fun getIceCandidates(
 @Query("session_id") sessionId: String,
 @Query("role") role: String
): IceCandidatesResponse

 @POST("api/live/end")
 suspend fun endSession(@Body body: EndSessionRequest): ApiMessageResponse
}

data class IceRequest(
 @SerializedName("session_id") val sessionId: String,
 val role: String,
 val candidate: String,
 @SerializedName("sdpMid") val sdpMid: String?,
 @SerializedName("sdpMLineIndex") val sdpMLineIndex: Int
)


```

```

data class IceCandidatesResponse(
 @SerializedName("session_id") val sessionId: String,
 val candidates: List<IceCandidateDto>
)
data class IceCandidateDto(
 val candidate: String,
 @SerializedName("sdpMid") val sdpMid: String?,
 @SerializedName("sdpMLineIndex") val sdpMLineIndex: Int
)
```
---
### Fix G — Remove Old PCM Code
Search and DELETE:
- Binary WebSocket send for mic audio (controller)
- Binary WebSocket receive + AudioTrack PCM playback (player)
- `live_start` handler expecting binary stream
Live audio = WebRTC remote track ONLY
---
### Verify Fix — Server Logs During Live Test
**Terminal 1 — coturn (MUST show activity after fix):**
```bash
sudo journalctl -u coturn -f
```
Expected after fix: lines with `session`, `allocation`, `user <schoolbell>`
**Terminal 2 — backend:**
```bash
docker compose logs -f api | grep -E "session|offer|answer|ice|player"
```
Log line	Meaning
`session_created`	Controller started OK
`player_notified sent=1`	Player WS online
`player_notified sent=0`	**Player WS offline — fix player app**
`offer_received`	Offer stored
`session_connected`	Answer received
Many ICE requests	Candidate exchange working
---
### Verify Fix — Android Logcat

```

Filter: `LiveWebRTC`

| Log | Must see |

|-----|

| `ice\_servers count=3` | Includes turn:194.233.86.3:3478 |

| `offer posted` | Controller |

| `answer posted` | Player |

| `ICE POST ok` | Both apps, multiple times |

| `ICE GET: N candidates` | N > 0 on both sides |

| `ICE state: CONNECTED` | Success |

Common Bugs !' Fixes

| Bug | Symptom | Fix |

|-----|

| Hardcoded STUN | coturn silent | Use API ice_servers |

| Gson `password` instead of `credential` | TURN auth fail | Fix IceServerDto field name |

| No ICE GET polling | ICE fail after answer | Add poll loop 500ms x 30s |

| Player skips live_session | No TURN on player | Store ice_servers on live_session |

| WS offline on player | sent=0, no answer | Foreground service + WS reconnect |

| Candidates before SDP | Some never added | Queue + flush pattern |

| Wrong ICE role | Empty GET response | controller role=controller, player role=player |

| Old PCM code still active | Conflicts / crash | Remove binary WS audio |

Which Repo Gets What

| File | Controller App | Player App |

|-----|

| LiveApiService.kt | session, offer, ice, end | config, answer, ice, end |

| LiveWebRtcController.kt | YES | NO |

| LiveWebRtcPlayer.kt | NO | YES |

| WS handler | live_answer, live_ice | live_session, live_offer, live_ice |

| Mic / Speaker | Mic capture | Speaker output |

Run this prompt **once in Controller repo, once in Player repo**.

Success Criteria

- [] Logcat shows `turn:194.233.86.3:3478` in ice_servers

- [] Logcat shows ICE POST + GET on both apps

- [] coturn journalctl shows allocation during live test

- [] Backend: session_created !' offer_received !' session_connected

- [] Controller UI: **"Live"**

- [] Player: mic audio on speaker automatically

- [] Works on different networks (WiFi + mobile data)

Đ

---Đ

Đ

Smart School Bell — ICE Fix App Prompt v2 Đ

*Server verified OK — fix Android app only*Đ